

Design Patterns Comportementaux en PHP

Objectif des patterns comportementaux

Les **design patterns comportementaux** s'intéressent à la **manière dont les objets interagissent entre eux**, à la **distribution des responsabilités**, et au **flux de contrôle** dans une application.

Ils facilitent la **communication entre objets** sans couplage fort.

Sommaire

1. [Observer](#)
2. [Strategy](#)
3. [Command](#)
4. [State](#)
5. [Iterator](#)
6. [Template Method](#)
7. [Chain of Responsibility](#)
8. [Mediator](#)
9. [Visitor](#)

1. Observer

 But :

Notifie automatiquement un groupe d'objets **quand un autre objet change d'état**.

 Exemple en PHP :

```
interface Observateur {
    public function mettreAJour(string $etat);
}

class Utilisateur implements Observateur {
    public function mettreAJour(string $etat) {
        echo "Notification reçue : $etat\n";
    }
}

class Sujet {
    private $observateurs = [];

    public function ajouter(Observateur $o) {
        $this->observateurs[] = $o;
    }
}
```

```

        public function notifier(string $etat) {
            foreach ($this->observateurs as $o) {
                $o->mettreAJour($etat);
            }
        }
    }

// Utilisation
$sujet = new Sujet();
$sujet->ajouter(new Utilisateur());
$sujet->notifier("Nouveau message");

```

2. 🧠 Strategy

📌 But :

Permet d'**interchanger dynamiquement des algorithmes** à l'exécution.

📦 Exemple en PHP :

```

interface TriStrategy {
    public function trier(array $donnees): array;
}

class TriAscendant implements TriStrategy {
    public function trier(array $donnees): array {
        sort($donnees);
        return $donnees;
    }
}

class Contexte {
    private $strategie;

    public function __construct(TriStrategy $s) {
        $this->strategie = $s;
    }

    public function executer(array $d): array {
        return $this->strategie->trier($d);
    }
}

// Utilisation
$contexte = new Contexte(new TriAscendant());
print_r($contexte->executer([3,1,2]));

```

3. 🎮 Command

📌 But :

Encapsule une **action dans un objet**, pour pouvoir l'exécuter, l'annuler, la mettre en file...

📦 Exemple en PHP :

```
interface Commande {
    public function executer();
}

class Lumiere {
    public function allumer() { echo "Lumière allumée\n"; }
}

class AllumerLumiere implements Commande {
    private $lumiere;

    public function __construct(Lumiere $l) {
        $this->lumiere = $l;
    }

    public function executer() {
        $this->lumiere->allumer();
    }
}

class Telecommande {
    public function envoyerCommande(Commande $c) {
        $c->executer();
    }
}

// Utilisation
$t = new Telecommande();
$t->envoyerCommande(new AllumerLumiere(new Lumiere()));
```

4. 🔄 State

📌 But :

Permet à un objet de **changer dynamiquement son comportement** selon son **état interne**.

📦 Exemple en PHP :

```
interface Etat {
    public function action();
}
```

```

class EtatRepos implements Etat {
    public function action() { echo "Je me repose\n"; }
}

class EtatTravail implements Etat {
    public function action() { echo "Je travaille\n"; }
}

class Personne {
    private Etat $etat;

    public function setEtat(Etat $e) {
        $this->etat = $e;
    }

    public function agir() {
        $this->etat->action();
    }
}

// Utilisation
$p = new Personne();
$p->setEtat(new EtatRepos());
$p->agir();
$p->setEtat(new EtatTravail());
$p->agir();

```

5. Iterator

 But :

Permet de **parcourir une collection sans exposer sa structure interne**.

 Exemple PHP (interface native) :

```

class Collection implements IteratorAggregate {
    private $elements = [];

    public function ajouter($valeur) {
        $this->elements[] = $valeur;
    }

    public function getIterator(): Traversable {
        return new ArrayIterator($this->elements);
    }
}

// Utilisation
$c = new Collection();

```

```
$c->ajouter("A");
$c->ajouter("B");

foreach ($c as $e) {
    echo $e . "\n";
}
```

6. Template Method

 But :

Définit **un squelette d'algorithme**, dont certaines étapes peuvent être personnalisées par des sous-classes.

 Exemple en PHP :

```
abstract class Recette {
    public function preparer() {
        $this->chauffer();
        $this->cuire();
        $this->servir();
    }

    abstract protected function cuire();
    protected function chauffer() { echo "Chauffage...\n"; }
    protected function servir() { echo "Servi !\n"; }
}

class RecettePizza extends Recette {
    protected function cuire() { echo "Cuisson pizza\n"; }
}

$recette = new RecettePizza();
$recette->preparer();
```

7. Chain of Responsibility

 But :

Passe une requête le long d'une chaîne d'objets jusqu'à ce qu'un objet la traite.

 Exemple en PHP :

```
abstract class Handler {
    protected ?Handler $suivant = null;
```

```

    public function suivant(Handler $h): Handler {
        $this->suivant = $h;
        return $h;
    }

    abstract public function gerer(string $type): ?string;
}

class AuthHandler extends Handler {
    public function gerer(string $type): ?string {
        if ($type === "auth") return "Authentification OK";
        return $this->suivant?->gerer($type);
    }
}

class LogHandler extends Handler {
    public function gerer(string $type): ?string {
        if ($type === "log") return "Log effectué";
        return $this->suivant?->gerer($type);
    }
}

// Chaîne
$auth = new AuthHandler();
$log = new LogHandler();
$auth->suivant($log);

// Test
echo $auth->gerer("log");

```

8. Mediator

 But :

Centralise la **communication entre objets** pour réduire leur couplage direct.

 Exemple en PHP :

```

class ChatRoom {
    public function envoyer($de, $message) {
        echo "$de dit : $message\n";
    }
}

class Utilisateur {
    private $nom;
    private $chat;

    public function __construct($nom, ChatRoom $c) {
        $this->nom = $nom;
    }
}

```

```

        $this->chat = $c;
    }

    public function parler($msg) {
        $this->chat->envoyer($this->nom, $msg);
    }
}

$c = new ChatRoom();
$u = new Utilisateur("Alice", $c);
$u->parler("Bonjour !");

```

9. Visitor

 But :

Permet d'**ajouter des opérations** à des objets **sans modifier leur code**.

 Exemple en PHP :

```

interface Visitable {
    public function accepter(Visiteur $v);
}

class Livre implements Visitable {
    public function accepter(Visiteur $v) {
        $v->visiterLivre($this);
    }
}

class CD implements Visitable {
    public function accepter(Visiteur $v) {
        $v->visiterCD($this);
    }
}

interface Visiteur {
    public function visiterLivre(Livre $l);
    public function visiterCD(CD $c);
}

class Afficheur implements Visiteur {
    public function visiterLivre(Livre $l) {
        echo "Livre affiché\n";
    }
    public function visiterCD(CD $c) {
        echo "CD affiché\n";
    }
}

```

```
// Utilisation  
$affiche = new Afficheur();  
$l = new Livre();  
$l->accepter($affiche);
```